



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA CURSO
TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS INTEGRADO AO
ENSINO MÉDIO

FILIPPO PYLORIDIS CIOTTI
LUCAS LANDES XAVIER MARTINS
MATHEUS ROBERTO SANTOS PEREIRA
NICOLAS RODRIGUES GALLINA

TURMA 213

DOCUMENTAÇÃO DO PROJETO SABOR&CLIC
Projeto Integrador - 3º Bimestre

SÃO PAULO – SP

2026

FILIPPO PYLORIDIS CIOTTI
LUCAS LANDES XAVIER MARTINS
MATHEUS ROBERTO SANTOS PEREIRA
NICOLAS RODRIGUES GALLINA

TURMA 213

DOCUMENTAÇÃO DO PROJETO SABOR&CLIC
Projeto Integrador - 3º Bimestre

Documentação do Projeto Integrador do terceiro bimestre apresentado no curso Técnico em Desenvolvimento de sistemas integrado ao ensino médio do Instituto Federal de São Paulo.

Professor(a): Claudete de Oliveira Alves,

SÃO PAULO – SP

2026

SUMÁRIO

1 Introdução ao Projeto	5
2 Objetivos e Limites do Projeto	5
2.1 Objetivos da parte 1 do Projeto.....	6
2.2 Limites.....	6
3 Atores e Funcionalidades	7
4 Integração entre Disciplinas.....	8
4.1 Design, HTML e CSS	8
4.2 JavaScript.....	9
4.3 Python e Flask.....	9
4.4 Banco de Dados	9
4.5 Segurança	9
5 Fluxo Integrado do Sistema	10
6. Fluxo de Funcionamento da Sabor&Clic	11
7 Arquitetura da Aplicação	11
8 Ambiente de Programação	12
9 Banco de Dados e DER	13
9.1 Dicionário de Dados	14
9.1.1 Entidade USUARIO e Especializações	14
9.1.2 Entidade PEDIDO	16
9.1.3 Entidade-relacionamento ITEM_PEDIDO	16
9.1.4 Entidade PRATO	17
9.1.5 Entidade-Relacionamento FICHA_TECNICA	17
9.1.5 Entidade INGREDIENTE.....	17
9.1.6 Entidade VENDA.....	18
9.1.7 Entidade AGENDAMENTO	18
9.1.8 Entidade-relacionamento AGENDAMENTO_BANCADA	19
9.1.9 Entidade BANCADA	19
9.1.10 Tabela de Relacionamentos	19
10 Identidade Visual do Site	21
11 Prototipação do site com o Figma	22
11.1 Módulo do Cliente.....	23
11.1.1 Cardápio Digital.....	23
11.1.2 Carrinho e checkout	24
11.2 Módulo do Cozinheiro	25
11.2.1 Painel de controle (KDS).....	25
11.3 Módulo Administrativo	27
11.3.1 Gestão de Cardápio	27

11.3.2 Gestão de Cozinhas	27
11.3.3 Controle de estoque	28
11.3.4 Visão de pedidos	29
11.3.5 Relatório de desempenho	30
11.3.6 Gestão de usuários.....	31
11.3.7 Cardápio do administrador.....	32
11.3.8 Exemplo de uma página em Alto Contraste.....	33
12 Arquitetura Flask	34
13 Arquitetura Javascript Inicial	37
14 Visualização sobre construção do Sabor&Clic na API.....	39
14.1 Recursos do JavaScript	40
14.2 Recursos do Python e Flask	41
15 Definição dos Níveis de Acesso e Regras de Validação de Dados	42
16 Conclusões e Continuidade do Projeto	43
17 Referências	45

1 Introdução ao Projeto

O Sabor & Clic é um sistema de gestão desenvolvido para atender às necessidades de Dark Kitchens e espaços gastronômicos, oferecendo uma solução para organizar e facilitar diferentes atividades relacionadas à operação desses estabelecimentos. O projeto surge a partir da necessidade de substituir processos realizados de maneira manual, como o controle de pedidos, o gerenciamento do cardápio e a organização das atividades da cozinha.

A proposta consiste no desenvolvimento de uma aplicação web que permita a interação entre diferentes tipos de usuários, cada um com suas próprias responsabilidades dentro do sistema. Dessa forma, o projeto busca centralizar essas operações em um único ambiente, tornando o gerenciamento das informações mais organizado e proporcionando uma experiência mais prática para os usuários.

O desenvolvimento do Sabor & Clic é realizado de forma integrada entre as disciplinas do projeto, envolvendo áreas como programação, banco de dados, segurança e design. Na Parte 1, o foco está principalmente no planejamento, na estruturação inicial do sistema, na documentação e na construção dos primeiros elementos que servirão de base para a implementação completa nas etapas seguintes.

2 Objetivos e Limites do Projeto

O principal objetivo do Sabor & Clic é desenvolver um sistema de gestão voltado para Dark Kitchens e espaços gastronômicos, capaz de centralizar e organizar atividades relacionadas ao cardápio, aos pedidos e à produção. A proposta é substituir processos que podem ser realizados manualmente por uma solução informatizada, tornando o gerenciamento das informações mais organizado e contribuindo para a redução de erros durante a operação.

O sistema deverá permitir que cada tipo de usuário desempenhe suas respectivas funções de acordo com suas responsabilidades. O administrador terá controle sobre o catálogo, o cliente poderá consultar os produtos e realizar pedidos, enquanto o cozinheiro poderá acompanhar e atualizar o andamento da produção. Dessa forma, o projeto busca estabelecer um fluxo integrado entre os diferentes usuários e as etapas envolvidas na realização de um pedido.

Além de atender às necessidades funcionais do sistema, o projeto tem como objetivo integrar os conhecimentos das diferentes disciplinas envolvidas em seu

desenvolvimento. A aplicação deverá reunir uma interface planejada e acessível, uma estrutura de programação capaz de lidar com as interações e regras do sistema, uma base de dados organizada e mecanismos de segurança adequados ao controle das informações e dos usuários.

2.1 Objetivos da parte 1 do Projeto

Na Parte 1, o objetivo é estabelecer a estrutura inicial do projeto e definir como o sistema será organizado antes de sua implementação completa. Essa etapa contempla a elaboração da documentação, o desenvolvimento do protótipo navegável da interface, a construção do modelo conceitual do banco de dados e a criação das primeiras estruturas de programação.

Também faz parte dessa etapa o desenvolvimento inicial das rotas do Flask e de scripts JavaScript voltados à manipulação da interface, permitindo representar e simular parte do funcionamento que será desenvolvido posteriormente. Dessa forma, a Parte 1 funciona como uma preparação para a implementação completa prevista na segunda etapa do projeto.

2.2 Limites

A Parte 1 apresenta algumas limitações naturais por se tratar de uma etapa de estruturação e simulação, e não da versão final do sistema. Nesse momento, o front-end ainda não está completamente implementado nem integrado a uma API funcional. Os elementos desenvolvidos em JavaScript representam principalmente a lógica e o comportamento esperado da aplicação, podendo utilizar dados definidos diretamente no próprio código para simular situações que posteriormente serão realizadas com dados reais.

O mesmo ocorre com o back-end desenvolvido em Python e Flask. Embora já existam estruturas de código e uma definição inicial das rotas que farão parte do sistema, elas ainda possuem caráter principalmente conceitual e de preparação para a integração futura. As requisições entre o front-end e o back-end ainda não estão funcionando como ocorrerão na aplicação completa, pois a implementação executável e a integração efetiva serão desenvolvidas posteriormente.

Outra limitação está relacionada à interface. Nesta etapa, o projeto ainda se encontra no processo de prototipação e estruturação visual, portanto a implementação

completa em HTML e CSS ainda não representa necessariamente a versão final do sistema. O protótipo desenvolvido no Figma serve como referência para a futura construção da interface.

O banco de dados também ainda não está implementado como uma estrutura funcional. Na Parte 1, o trabalho está concentrado no modelo conceitual, representado pelo DER, e na documentação das informações que deverão compor o banco posteriormente. A implementação física e a persistência efetiva dos dados fazem parte da etapa seguinte.

Da mesma forma, os aspectos de segurança ainda estão em fase de definição. Nesta etapa são estabelecidos os níveis de acesso e as regras iniciais de validação, enquanto mecanismos mais completos de autenticação e proteção da aplicação serão implementados posteriormente.

Assim, a primeira parte do projeto não representa ainda o sistema final em funcionamento, mas sim sua base estrutural, conceitual e visual, permitindo que as diferentes partes do projeto sejam definidas antes da implementação completa.

3 Atores e Funcionalidades

O sistema Sabor & Clic possui três principais tipos de usuários: Administrador, Cliente e Cozinheiro. A divisão desses atores permite que cada usuário tenha responsabilidades relacionadas à sua função dentro do estabelecimento, evitando que todas as operações fiquem concentradas em um único perfil.

O Administrador é responsável pelo gerenciamento do catálogo do sistema. Ele poderá criar, editar e remover itens do cardápio, além de definir informações como preços e descrições e controlar quais pratos estarão disponíveis para visualização. Dessa maneira, é o administrador quem mantém as informações dos produtos atualizadas e determina o que estará disponível para os clientes.

O Cliente é o usuário que utiliza o sistema para consultar o cardápio e realizar seus pedidos. Ele poderá visualizar os pratos disponíveis, selecionar os itens desejados, montar seu carrinho e finalizar a compra. Após realizar um pedido, também poderá acompanhar seu andamento por meio dos status disponibilizados pelo sistema.

O Cozinheiro é responsável pelo acompanhamento da produção dos pedidos. Sua utilização do sistema estará relacionada principalmente à visualização da fila de

pedidos e ao acompanhamento do que precisa ser preparado. Ele poderá registrar informações relacionadas ao preparo e atualizar o status dos pedidos conforme eles avançam no processo de produção.

Ator	Função no Sistema	Principais Funcionalidades	Resultado de sua atuação
Administrador	Gerenciamento do catálogo	Criar, editar e remover pratos, definir preços e descrições, ativar ou desativar itens	Mantém o cardápio atualizado e controla os produtos disponíveis
Cliente	Consumo e realização de pedidos	Visualizar o cardápio; selecionar produtos; montar o carrinho; finalizar pedidos; acompanhar o andamento	Gera pedidos e acompanha sua situação
Cozinheiro	Operação produção	Visualizar pedidos pendentes; acompanhar a fila; registrar informações do preparo; atualizar o status	Organiza e atualiza o andamento da produção

Fonte: Elaboração da Equipe, 2026.

4 Integração entre Disciplinas

O Sabor & Clic depende da integração entre as diferentes disciplinas para que suas partes funcionem de maneira conjunta. Cada área possui uma responsabilidade própria, mas os resultados de uma disciplina servem como base para o trabalho das demais.

4.1 Design, HTML e CSS

A disciplina de Artes Visuais e Design será responsável pela definição da aparência e da experiência de utilização do sistema. Nessa etapa são definidos aspectos como identidade visual, organização das telas, tipografia, cores, hierarquia das informações, usabilidade e responsividade. O protótipo desenvolvido servirá como referência para a implementação posterior da interface.

O HTML e o CSS serão utilizados posteriormente para transformar essa estrutura visual em uma interface funcional, mantendo a organização e as características estabelecidas no protótipo.

4.2 JavaScript

O JavaScript ficará responsável pelo comportamento e pela interatividade da interface. Ele permitirá que as ações realizadas pelo usuário sejam processadas de forma dinâmica, realizando alterações nas informações apresentadas na tela e trabalhando com os dados utilizados pela aplicação.

Na Parte 1, essa lógica ainda pode ser representada por meio de dados e situações simuladas dentro do próprio código. Posteriormente, o JavaScript será integrado ao back-end para realizar a comunicação efetiva com o sistema.

4.3 Python e Flask

O Python, por meio do Flask, será responsável pela estrutura do back-end. O Flask terá a função de organizar as rotas da aplicação, receber as solicitações feitas pelo sistema e realizar o processamento necessário antes de devolver uma resposta.

Na Parte 1, as rotas e estruturas desenvolvidas ainda representam uma preparação para essa comunicação. A integração completa entre front-end e back-end será desenvolvida nas etapas posteriores.

4.4 Banco de Dados

O Banco de Dados será responsável pela organização e pelo armazenamento das informações utilizadas pelo sistema. Na primeira etapa, essa disciplina concentra-se na definição da estrutura conceitual dos dados por meio do DER e do dicionário de dados.

Posteriormente, essa estrutura será transformada em um banco de dados relacional funcional, que será utilizado pelo back-end para armazenar e consultar as informações da aplicação.

4.5 Segurança

A disciplina de Segurança será responsável por definir como o sistema deverá controlar o acesso às suas funcionalidades e proteger as informações manipuladas pela aplicação. Na Parte 1, serão estabelecidos principalmente os níveis de acesso e as regras iniciais de validação. Nas etapas seguintes, esses conceitos serão incorporados ao funcionamento efetivo do sistema.

5 Fluxo Integrado do Sistema

O funcionamento integrado do projeto começará pela interface, construída a partir das definições realizadas na área de Design, que estabelece a organização visual e a experiência de utilização do sistema. O HTML e o CSS transformarão essas definições em uma interface funcional, enquanto o JavaScript será responsável por controlar as interações realizadas pelo usuário e pelo comportamento dinâmico das páginas.

Quando uma ação realizada pelo usuário precisar ser processada pelo sistema, o JavaScript será responsável por organizar as informações e realizar a comunicação com o back-end. Essa comunicação ocorrerá por meio de requisições assíncronas utilizando a Fetch API, com os dados sendo transmitidos em JSON. O projeto estabelece justamente essa separação entre a interface e a camada responsável pelo processamento das informações.

As requisições serão recebidas pelo Flask, que funcionará como a camada intermediária responsável pelas rotas e pelo processamento das informações. O Python ficará responsável pelas regras de negócio e pelos procedimentos necessários para que cada solicitação seja tratada corretamente. Quando for necessário consultar ou alterar informações armazenadas, o Flask realizará a comunicação com o Banco de Dados, mantendo essa conexão concentrada no back-end.

Depois que o processamento for concluído, o back-end enviará uma resposta para a interface, novamente utilizando uma estrutura de dados adequada à comunicação entre as duas partes. O JavaScript poderá então interpretar essa resposta e atualizar a interface de acordo com o resultado da operação, permitindo que as informações sejam modificadas dinamicamente sem depender necessariamente de um novo carregamento da página.

A Segurança estará integrada a esse fluxo, acompanhando o processamento das solicitações para verificar as permissões dos usuários e validar as informações recebidas. Assim, o sistema não dependerá apenas da interface para determinar o que um usuário pode fazer, mantendo essas verificações também na camada responsável pelo processamento.

6. Fluxo de Funcionamento da Sabor&Clic

7 Arquitetura da Aplicação

O projeto adotará uma arquitetura organizada em camadas, buscando separar as responsabilidades de cada parte da aplicação e facilitar sua manutenção e evolução. No lado do cliente, a interface será estruturada com HTML, CSS e JavaScript, responsáveis pela apresentação das informações e pelas interações realizadas pelo usuário. No back-end, o Flask será utilizado para organizar o funcionamento da aplicação, contando com Blueprints para dividir e estruturar as rotas de acordo com os diferentes módulos e funcionalidades do sistema. Essa organização permite que as rotas sejam agrupadas de forma lógica, evitando que toda a aplicação fique concentrada em um único arquivo.

As requisições recebidas pelo servidor serão direcionadas pelas rotas aos respectivos Controllers, que serão responsáveis por realizar as validações e executar os casos de uso. Os Models representarão os dados utilizados pela aplicação e, durante a Parte 1, poderão trabalhar com dados demonstrativos mantidos em memória. Em uma etapa posterior, esses dados serão substituídos pela persistência em um banco de dados relacional, que será acessado exclusivamente pelo back-end.

Camada	Responsabilidade	Exemplos
View / Interface	Apresentar as informações do sistema e permitir a interação do usuário.	Cardápio, formulários e painel de pedidos
JavaScript	Controlar o comportamento da interface e o gerenciamento das informações apresentadas ao usuário.	Carrinho, filtros e atualizações da tela
Routes / Blueprints Flask	Organizar e direcionar as requisições para as funcionalidades correspondentes da aplicação.	Rotas relacionadas a pedidos, produtos e usuários

Controllers	Processar as solicitações, realizar validações e executar as regras associadas aos casos de uso.	Validação de um pedido e processamento de uma operação
Models	Representar e manipular os dados utilizados pela aplicação.	Usuário, produto, pedido e demais entidades
Banco de Dados	Armazenar de forma persistente e organizada os dados do sistema.	Registros, relacionamentos e restrições do banco

Fonte: Elaboração da Equipe, 2026.

A organização do projeto busca manter o front-end e o back-end separados por responsabilidades, permitindo que cada parte seja desenvolvida e posteriormente integrada sem comprometer a estrutura das demais. No back-end, a utilização de Blueprints possibilitará organizar as rotas em módulos independentes, agrupando funcionalidades relacionadas e tornando a estrutura do Flask mais clara e escalável à medida que novas funcionalidades forem adicionadas.

Durante a Parte 1, algumas dessas camadas ainda funcionarão de maneira independente e utilizarão dados simulados para representar o comportamento esperado do sistema. As estruturas de rotas, Controllers e Models serão desenvolvidas como base para a aplicação, enquanto a persistência definitiva dos dados será implementada em uma etapa posterior. Dessa forma, a arquitetura definida nesta etapa estabelece a organização necessária para que, na versão final, as diferentes camadas possam funcionar de maneira integrada e consistente.

8 Ambiente de Programação

Para o desenvolvimento do projeto, será utilizado um ambiente virtual Python, por meio do venv, com o objetivo de organizar e isolar as dependências utilizadas pela aplicação. Dessa forma, os pacotes necessários ao funcionamento do sistema poderão ser instalados dentro do ambiente virtual, evitando conflitos com outras instalações ou projetos presentes no computador.

Nesse ambiente serão instaladas as bibliotecas e ferramentas necessárias para o desenvolvimento do back-end, como o Flask, além de outros pacotes que possam

ser utilizados ao longo do projeto. O SQLite3, por sua vez, será utilizado para o gerenciamento do banco de dados, integrando-se à aplicação por meio do Python. Essa organização permitirá manter as dependências do projeto centralizadas e facilitará sua configuração e execução em diferentes ambientes.

9 Banco de Dados e DER

O banco de dados do Sabor & Clic será responsável por organizar e armazenar as principais informações utilizadas pelo sistema, estabelecendo as relações entre usuários, agendamentos, pedidos, vendas, pratos, ingredientes e bancadas. Para representar essa estrutura inicialmente, foi elaborado o Diagrama Entidade-Relacionamento (DER), permitindo visualizar as entidades que fazem parte do sistema, seus atributos e os relacionamentos existentes entre elas.

A estrutura parte da entidade Usuário, que representa a base para os diferentes perfis existentes no sistema. A partir dela, são definidas especializações que representam os tipos de usuário: Cliente, Cozinheiro e Administrador. Cada especialização possui características e responsabilidades próprias dentro do sistema, mantendo em comum as informações básicas de um usuário.

A partir do Cliente, é possível estabelecer o fluxo de utilização do sistema. O cliente pode realizar um agendamento, relacionando-se com uma bancada disponível para determinado período. O agendamento, por sua vez, está relacionado a uma venda, representando a transação associada à utilização do serviço. Além disso, o sistema contempla o fluxo de pedidos, que são preparados pelo cozinheiro e possuem os respectivos itens solicitados pelo cliente.

Os pedidos são compostos por itens, que estão relacionados aos pratos disponíveis no sistema. Cada prato possui uma ficha técnica, responsável por representar os ingredientes necessários para sua preparação e as informações relacionadas ao seu uso. A estrutura de ingredientes também considera a possibilidade de um ingrediente ser substituído por outro, permitindo representar alternativas que podem ser utilizadas na preparação dos pratos.

Dessa forma, o modelo busca representar o funcionamento do sistema desde seus usuários e suas especializações, passando pelos agendamentos e vendas, até chegar ao pedido, preparo, pratos e ingredientes. A partir desse modelo conceitual, foi elaborado o Dicionário de Dados, detalhando cada entidade, seus atributos e suas

respectivas características, seguindo a sequência lógica estabelecida no DER.

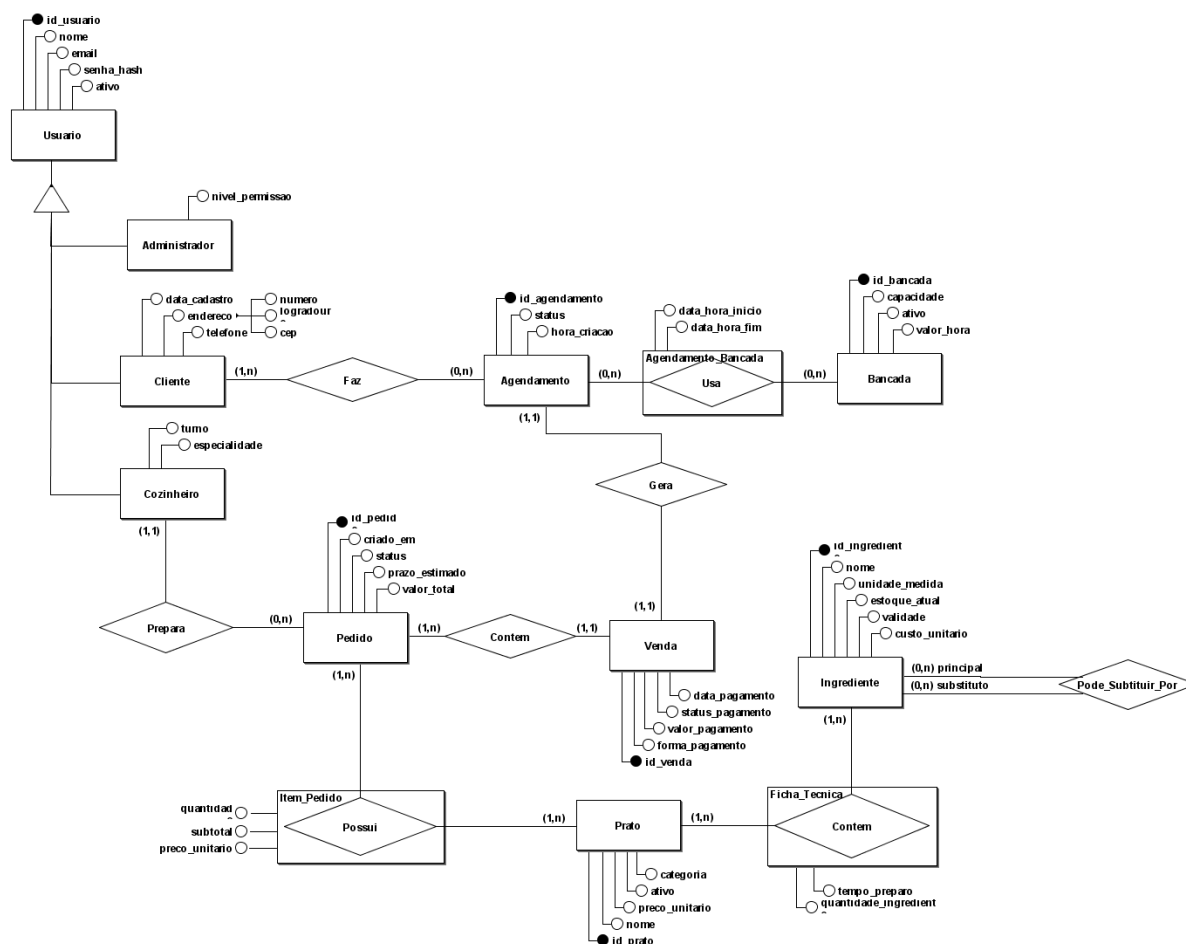


Figura 1 - DER do Sabor&Clic

Fonte: Elaborada pela Equipe, 2026

9.1 Dicionário de Dados

O dicionário de dados apresenta, de forma organizada, as informações que compõem as entidades e relacionamentos definidos para o banco de dados do Sabor & Clic. Nele, são detalhados os atributos de cada estrutura, permitindo compreender a finalidade de cada campo e sua função dentro do modelo de dados. Essa documentação complementa o DER e auxilia na definição e padronização das informações que serão utilizadas na implementação do sistema.

9.1.1 Entidade USUARIO e Especializações

Esta tabela junta todas as informações referentes a um mesmo usuário que está cadastrado no sistema:

Atributos	Tipo do Atributo	Descrição e Domínio
id_usuario	INT	identificador único, obrigatório e positivo(PK)
nome	VARCHAR(150)	Nome completo do usuário, não nulo
email	VARCHAR(150)	Email do usuário, não nulo
senha_hash	VARCHAR(150)	Senha da conta do usuário em formato hash(nunca salvar em texto puro), não nulo
ativo	BOOLEAN	Indicação se o acesso está habilitado

Fonte: Elaboração da Equipe, 2026.

Cada usuário está separado em 3 especializações (ADMINISTRADOR, CLIENTE e COZINHEIRO), onde cada uma especialização tem seus atributos. A exclusividade ou sobreposição dos perfis deve ser definida na implementação, pois não está explicitada no diagrama. O Administrador não tem relacionamentos devido às suas funcionalidades de regra de negócio, serem parte da aplicação definir, uma vez que os seus dados não são necessários para outra aplicação no DER, além de definir seu nível de permissão

Atributos	Tipo do Atributo	Descrição e Domínio
Administrador.nivel_de_permissao	INT	Níveis de permissão modelo CRUD
Cliente.data_cadastro	DATETIME	Data da criação da conta
Cliente.endereco	VARCHAR(200)	Endereço do cliente, pode ser nulo em caso de alugar a bancada
Cliente.telefone	INT	Telefone de contato do cliente, não nulo
Cozinheiro.turno	ENUM(MANHÃ, TARDE, NOITE)	Turno de trabalho, não nulo
Cozinheiro.especialidade	VARCHAR(200)	Especialização culinária do cozinheiro, não nulo

Fonte: Elaboração da Equipe, 2026.

9.1.2 Entidade PEDIDO

A entidade PEDIDO agrupa os pratos associados a cada pedido, permitindo registrar o valor total da compra e uma estimativa do tempo necessário para o preparo.

Atributos	Tipo do Atributo	Descrição e Domínio
id_pedido	INT	Identificador único, obrigatório e positivo(PK)
criado_em	DATETIME	Data de criação do pedido
status	ENUM(PRONTO, EM PREPARO, ENTREGUE, CANCELADO)	Status do pedido
prazo_estimado	DATETIME	Estimativa de quando o pedido vai ficar pronto
valor_total	DECIMAL(10,2)	Valor do pedido, não nulo e positivo

Fonte: Elaboração da Equipe, 2026.

9.1.3 Entidade-relacionamento ITEM_PEDIDO

A entidade ITEM_PEDIDO registra os pratos associados a cada pedido, armazenando a quantidade e o valor de cada prato no momento da realização da compra.

Atributos	Tipo do Atributo	Descrição e Domínio
quantidade	INT	quantidade de cada prato no pedido
subtotal	DECIMAL(10,2)	Soma do preço de todos os pratos vezes a quantidade de cada um, positivo e não nulo
preco_unitario	DECIMAL(10,2)	Valor individual de cada prato, positivo e não nulo

Fonte: Elaboração da Equipe, 2026.

9.1.4 Entidade PRATO

A entidade PRATO armazena as informações de cada prato, como nome, preço e demais características necessárias para sua identificação e comercialização.

Atributos	Tipo do Atributo	Descrição e Domínio
id_prato	INT	Identificador único, obrigatório e positivo(PK)
nome	VARCHAR(100)	Nome do prato, não nulo
preco_unitario	DECIMAL(10,2)	Preço unitário do prato, não nulo e positivo
ativo	BOOLEAN	Disponibilidade no cardápio
categoria	VARCHAR(100)	Categoria do prato

Fonte: Elaboração da Equipe, 2026.

9.1.5 Entidade-Relacionamento FICHA_TECNICA

A entidade FICHA_TECNICA relaciona os pratos aos seus ingredientes, registrando a quantidade utilizada e o tempo de preparo correspondente a cada ingrediente.

Atributos	Tipo do Atributo	Descrição e Domínio
quantidade_ingrediente	INT	Quantidade de cada ingrediente necessário
tempo_preparo	DATETIME	tempo de preparo médio, positivo e não nulo

Fonte: Elaboração da Equipe, 2026.

9.1.5 Entidade INGREDIENTE

A entidade INGREDIENTE armazena as informações dos ingredientes utilizados no preparo dos pratos, incluindo seu estoque e demais dados de controle.

Atributos	Tipo do Atributo	Descrição e Domínio
id_ingrediente	INT	Identificador único, obrigatório e positivo(PK)
nome	VARCHAR(100)	Nome do ingrediente, não nulo
unidade_medida	VARCHAR(20)	Unidade de medida, ex: kg, g, L, ml

estoque_atual	DECIMAL(10,2)	Quantidade disponível no estoque
validade	DATE	Data de vencimento do ingrediente, não nulo
custo_unitario	DECIMAL(10,2)	Preço unitário de cada ingrediente, não nulo

Fonte: Elaboração da Equipe, 2026.

9.1.6 Entidade VENDA

A entidade PEDIDO registra as informações da venda, incluindo o valor total do pedido, a forma de pagamento, a data do pagamento e outros dados relacionados à sua realização.

Atributos	Tipo do Atributo	Descrição e Domínio
id_venda	INT	Identificador único, obrigatório e positivo(PK)
forma_pagamento	ENUM(DINNHEIRO, DEBITO, CREDITO, PIX)	Método de pagamento da venda, não nulo
valor_pagamento	DECIMAL(10,2)	Custo da venda, não nulo e positivo
status_pagamento	BOOLEAN	O pagamento foi realizado ou não
data_pagamento	DATETIME	Quando o pagamento foi efetuado

Fonte: Elaboração da Equipe, 2026.

9.1.7 Entidade AGENDAMENTO

A entidade AGENDAMENTO registra a data desejada pelo cliente para alugar a bancada, e outros dados relacionados ao agendamento do cliente.

Atributos	Tipo do Atributo	Descrição e Domínio
id_agendamento	INT	Identificador único, obrigatório e positivo(PK)
status	BOOLEAN	Disponibilidade da data
hora_criacao	DATETIME	Data que foi feito o agendamento

Fonte: Elaboração da Equipe, 2026.

9.1.8 Entidade-relacionamento AGENDAMENTO_BANCADA

A entidade de relacionamento AGENDAMENTO_BANCADA relaciona a data desejada pelo cliente com a disponibilidade da bancada.

Atributos	Tipo do Atributo	Descrição e Domínio
data_hora_inicio	DATETIME	Horário de início do agendamento da bancada, não nulo
data_hora_fim	DATETIME	Horário do fim do agendamento da bancada, não nulo

Fonte: Elaboração da Equipe, 2026.

9.1.9 Entidade BANCADA

A entidade BANCADA armazena informações como sua capacidade e sua disponibilidade para agendamento.

Atributos	Tipo do Atributo	Descrição e Domínio
id_bancada	INT	Identificador único, obrigatório e positivo(PK)
capacidade	TINYINT	Capacidade máxima de pessoas na bancada
ativo	BOOLEAN	Disponibilidade da bancada
valor_hora	DECIMAL(10,2)	Custo por hora da bancada, positivo e não nulo

Fonte: Elaboração da Equipe, 2026.

9.1.10 Tabela de Relacionamentos

A tabela apresenta os relacionamentos entre as entidades do sistema, identificando o nome de cada relacionamento e suas respectivas cardinalidades. Ela permite visualizar como as entidades se associam dentro do sistema e a quantidade de registros que podem estar relacionados entre si. Dessa forma, são apresentadas as relações de 1:1, 1:N e N:N, conforme as necessidades e regras estabelecidas para cada entidade.

Relacionamento	Entidades	Cardinalidade e mapeamento
FAZ	CLIENTE → AGENDAMENTO	Cliente: um ou vários agendamentos. Agendamento: zero ou vários clientes.
USA	AGENDAMENTO ↔ BANCADA	Agendamento: zero a vários. Bancada: zero a vários. Relacionamento N:N, com a entidade associativa Agendamento_Bancada, que possui data/hora de início e fim.
GERA	AGENDAMENTO ↔ VENDA	Um agendamento gera exatamente uma venda e uma venda está ligada a exatamente um agendamento. Relacionamento 1:1.
PREPARA	COZINHEIRO → PEDIDO	Cozinheiro: pode preparar zero a vários pedidos. Pedido: é preparado por um cozinheiro.
CONTÉM	PEDIDO → VENDA	Pedido: possui uma ou várias vendas. Venda: pertence a um único pedido.
POSSUI	PEDIDO ↔ PRATO	Pedido pode possuir um ou vários pratos, e um prato pode estar em um ou vários pedidos. Relação N:N realizada por ITEM_PEDIDO, que possui quantidade, subtotal e preço unitário.
FICHA_TÉCNICA	PRATO ↔ INGREDIENTE	Um prato possui um ou vários ingredientes, e um ingrediente pode estar em um ou vários pratos. Relação N:N realizada por FICHA_TECNICA, com quantidade do ingrediente e tempo de preparo.

PODE_SUBSTITUIR_POR	INGREDIENTE ↔ INGREDIENTE	Um ingrediente pode substituir zero ou vários outros ingredientes, e também pode ser substituído por zero ou vários ingredientes. Relação recursiva N:N.
---------------------	------------------------------	--

10 Identidade Visual do Site

A seleção tipográfica do projeto foi projetada estrategicamente para transmitir modernidade, agilidade e conforto visual em diferentes dispositivos. A família escolhida foi a Poppins, utilizando os pesos Regular (400), Medium (500), SemiBold (600) e Bold (700). Essa fonte geométrica sem serifa oferece excelente legibilidade em telas digitais de todos os tamanhos, enquanto seus traços uniformes e limpos evitam a distorção visual em ambientes de alta rotação, como cozinhas e caixas, além de transmitirem uma estética moderna e acolhedora. A hierarquia visual organiza-se de forma dinâmica: os títulos principais utilizam o peso 700 em 32px para orientar o usuário imediatamente sobre o contexto da página e criar um ponto focal de entrada visual. Os subtítulos e títulos de cards variam de 16px a 20px com pesos 600 a 700 para estruturar blocos de informação de forma modular, facilitando a leitura dinâmica e a varredura visual rápida. Já o corpo de texto e os parágrafos, dimensionados entre 13px e 15px com pesos 400 a 500, garantem conforto em leituras prolongadas e previnem a fadiga visual dos operadores do sistema.

No que diz respeito à identidade visual e ao sistema de cores, o projeto adota uma abordagem duplo-temática para atender tanto à identidade de marca e ao conforto cotidiano quanto às exigências severas de acessibilidade. No modo normal, voltado à identidade e ao acolhimento, a cor primária de destaque é o Laranja Vibrante (#F06C2D), escolhido por estimular o apetite, transmitir dinamismo e atuar como uma âncora visual para direcionar a atenção aos botões de ação e preços, acelerando a tomada de decisão. O fundo principal em tom Creme (#FFF8EB) substitui o branco estéril para proporcionar uma sensação de aconchego e proximidade típica de ambientes gastronômicos tradicionais, reduzindo o brilho excessivo da tela e o cansaço ocular. Elementos estruturais e superfícies utilizam o branco (#FFFFFF) em cartões e blocos para criar profundidade limpa, enquanto textos e suportes em cinza

(#232323 e variações) garantem contraste legível sem o impacto agressivo do preto puro, complementados por indicadores suaves de status para comunicar o estado do estoque ou conclusão de tarefas sem poluição visual.

Em contrapartida, o modo de alto contraste prioriza a acessibilidade estendida por meio de um fundo absoluto (#000000) em todas as superfícies, o que elimina ambiguidades visuais, ruídos de fundo e conflitos de estilização dinâmica para criar uma base neutra e definitiva. A tipografia de alto contraste combina branco puro e tons claros (#FFFFFF e #CCCCCC) para eliminar barreiras de legibilidade e atender aos critérios mais rigorosos para deficiências visuais severas. A cor de destaque interativo utiliza o Amarelo Luminoso (#FFFF00), que oferece o maior índice de percepção humana e contraste cromático em fundos escuros, destacando instantaneamente elementos interativos, bordas de foco e valores financeiros sem esforço visual. Além disso, a demarcação estrutural emprega contornos brancos e nítidos para delimitar caixas, tabelas e campos de formulário, compensando a ausência de sombras e texturas difusas.

O guia de diretrizes visuais (Design System) estabelece que os componentes de ação e botões sejam projetados com cantos arredondados para suavizar a interface, contando com transições responsivas que fornecem feedback imediato de toque ou clique. Os campos de entrada e formulários foram desenvolvidos com áreas de respiro confortáveis, bordas de alta definição e marcações visuais evidentes para o estado de foco, orientando o preenchimento sem margem para erros, enquanto a estruturação modular em cards organiza-se em grades flexíveis que se adaptam de forma fluida a diferentes proporções de tela.

11 Prototipação do site com o Figma

Nesta etapa, são apresentadas as principais páginas que compõem o sistema, desenvolvidas e organizadas no Figma. A visualização permite compreender a estrutura de cada tela, sua organização visual e a forma como os elementos são distribuídos ao longo da interface.

A primeira página apresentada é a tela de login, que introduz o usuário à identidade visual da marca e ao ambiente do sistema. Nela, é realizada a autenticação de usuários e colaboradores, estabelecendo o primeiro contato com a plataforma e direcionando o acesso de acordo com o perfil do usuário.

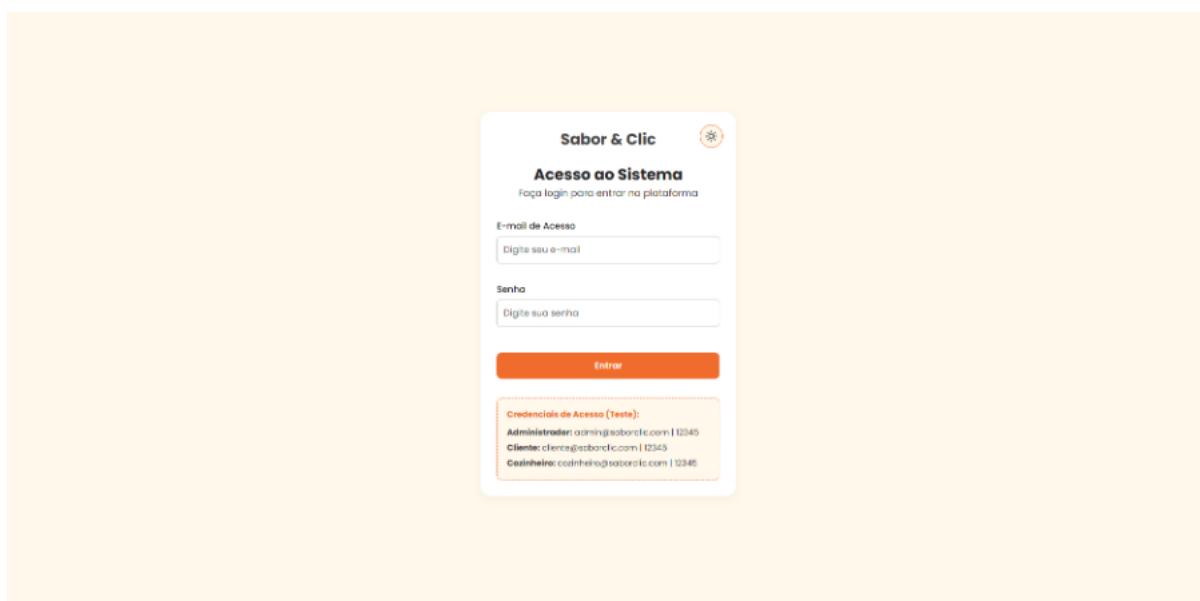


Figura: Página de login da Sabor&Clic.

Fonte: Elaborada pela Equipe, 2026.

A partir da tela de login, o usuário poderá, de acordo com o resultado da autenticação, acessar o painel correspondente ao seu perfil: cliente, administrador ou cozinheiro.

11.1 Módulo do Cliente

A seguir, são apresentadas as páginas que compõem o módulo do Cliente, voltadas à navegação pelo sistema e às funcionalidades disponíveis para esse perfil.

11.1.1 Cardápio Digital

Esta página é responsável pela interface de visualização e seleção de itens, estruturada para garantir uma navegação ágil entre o catálogo e o carrinho. Nela, o usuário pode visualizar os produtos disponíveis, conferir informações como nome, descrição e preço, além de adicionar ou remover itens do carrinho de forma prática e organizada, otimizando a experiência do usuário.

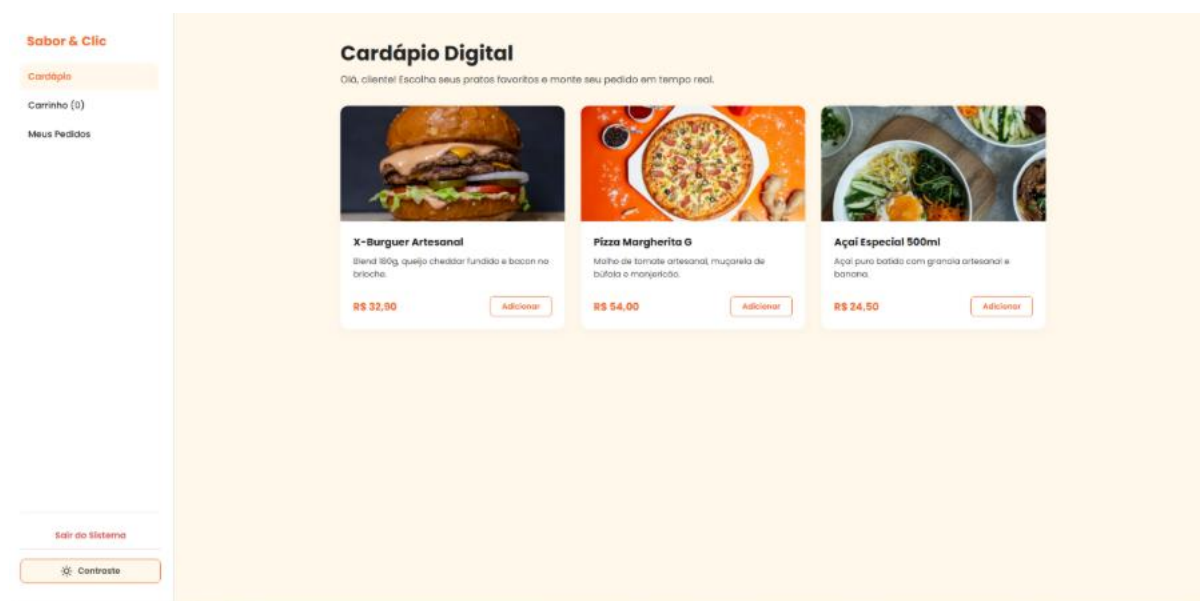


Figura: Página de cardápio da Sabor&Clic.

Fonte: Elaborado pela Equipe, 2026.

11.1.2 Carrinho e checkout

Página responsável pela gestão dos itens escolhidos e pela finalização do pedido. Nela, o usuário pode conferir os produtos selecionados, visualizar as quantidades e valores e revisar o pedido antes de concluir a compra, garantindo um processo simples e organizado.



Figura: Página de carrinho da Sabor&Clic.

Fonte: Elaborado pela Equipe, 2026.

Rastreamento de pedido

Página responsável pelo acompanhamento do status do pedido, permitindo que o usuário visualize o andamento da solicitação desde a confirmação até a finalização da entrega, de forma clara e organizada.

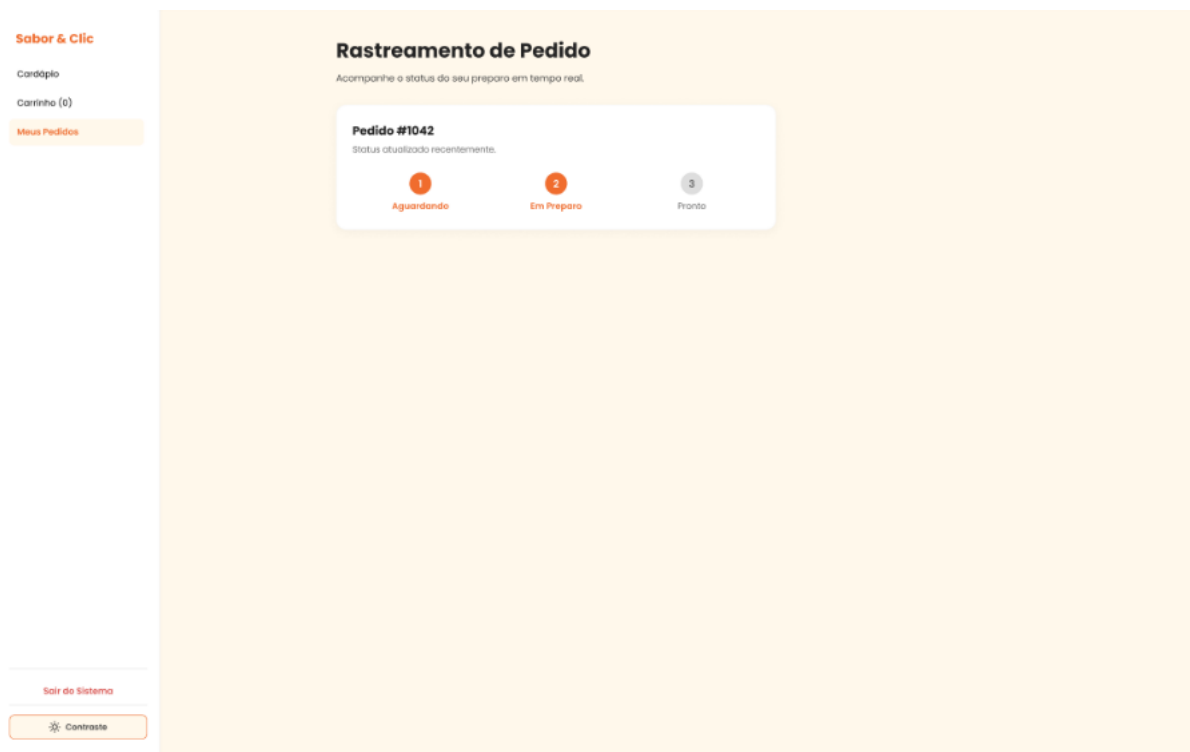


Figura: Página de rastreamento da Sabor&Clic.

Fonte: Elaborado pela Equipe, 2026.

11.2 Módulo do Cozinheiro

A seguir, são apresentadas as páginas que compõem a área do Cozinheiro, destinadas ao acompanhamento e gerenciamento dos pedidos durante o processo de preparo.

11.2.1 Painel de controle (KDS)

Interface responsável pelo monitoramento do andamento dos pedidos e pelo controle do preparo pela equipe de cozinha. Permite acompanhar os pedidos em diferentes etapas, facilitando a organização e o despacho das solicitações.



Figura: Página do painel de controle.

Fonte: Elaborado pela Equipe, 2026.

11.2.2 Cardápio do cozinheiro

O cozinheiro tem acesso ao cardápio disponível da loja para consultar os itens e suas informações, porém não possui permissão para realizar pedidos ou efetuar compras pelo sistema.

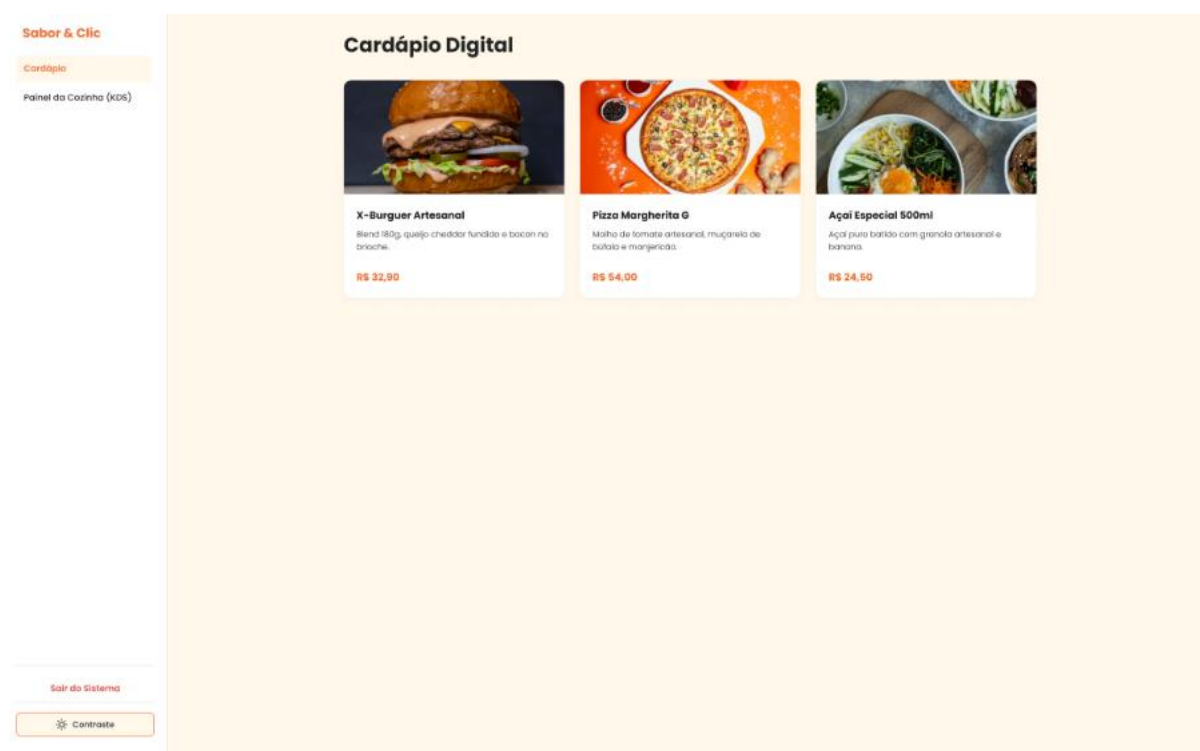


Figura: Página do cardápio do cozinheiro.

Fonte: Elaborado pela Equipe, 2026

11.3 Módulo Administrativo

A seguir, são apresentadas as páginas que compõem a área do Administrador, voltadas ao gerenciamento e controle das funcionalidades administrativas do sistema.

11.3.1 Gestão de Cardápio

Página responsável pelo controle dos itens disponíveis, permitindo organizar produtos, definir preços e classificá-los por categorias de forma simples e organizada.

A imagem mostra a interface do usuário para o Painel do Administrador, especificamente a seção de Gestão de Cardápio. O layout é dividido em uma barra lateral esquerda e uma área principal de conteúdo.

Barra Lateral Esquerda:

- Logo: Sabor & Cilec
- Menu: Cardápio do cliente
- Menu: ADMINISTRAÇÃO
 - Gerenciar Cardápio
 - Pedidos Gerais
 - Gerenciar Usuários
 - Gestão de Cozinhas
 - Estoque de Alimentos
 - Relatório de Desempenho
 - Painel da Cozinha (KDS)
- Botão: Sair do Sistema
- Botão: Contraste

Área Principal de Conteúdo:

- Título: Painel do Administrador
- Subtítulo: Cadastre, edite ou remova os pratos disponíveis no cardápio do sistema.
- Formulário: Adicionar / Editar Prato
 - Nome do Prato: Campo de texto.
 - Categoria: Campo de texto com o valor "Lanches" selecionado.
 - Preço (R\$): Campo de texto.
 - Descrição: Campo de texto.
 - Botão: Cadastrar Prato (em laranja).
- Seção: Pratos Cadastrados (abaixo do formulário).

Figura: Página do painel do administrador

Fonte: Elaborado pela Equipe, 2026

11.3.2 Gestão de Cozinhas

Página responsável pela configuração e organização dos setores e áreas de preparo da loja, permitindo definir e gerenciar os espaços utilizados pela equipe durante a produção dos pedidos.



Figura: Página da gestão de cozinhas e bancadas

Fonte: Elaborado pela Equipe, 2026

11.3.3 Controle de estoque

Página responsável pelo monitoramento dos insumos disponíveis, permitindo acompanhar a quantidade em estoque, a volumetria e possíveis alertas críticos, facilitando o controle e a reposição dos produtos.

Sabor & Clic

Cardápio do cliente

ADMINISTRAÇÃO

Gerenciar Cardápio

Pedidos Gerais

Gerenciar Usuários

Gestão de Cozinhas

Estoque de Alimentos

Relatório de Desempenho

Panela da Cozinha (KDS)

Sair do Sistema

Contraste

Controle de Estoque de Alimentos

Monitore os insumos disponíveis para a produção dos pratos em tempo real.

Insumo / ingrediente	Categoria	Quantidade Atual	Status	Ações
Blend de Carne Bovina (150g)	Carnes	45 unidades	Normal	+ Repor
Queijo Cheddar fatiado	Laticínios	8 pacotes	Estoque Baixo	+ Repor
Pão Brioche artesanal	Panificação	12 unidades	Estoque Baixo	+ Repor
Molho Especial da Casa	Molhos	30 frascos	Normal	+ Repor

Figura: Página do estoque de alimentos

Fonte: Elaborado pela Equipe, 2026

11.3.4 Visão de pedidos

Página responsável pelo acompanhamento geral das operações comerciais, permitindo visualizar e controlar as principais atividades e informações do negócio de forma centralizada e organizada.

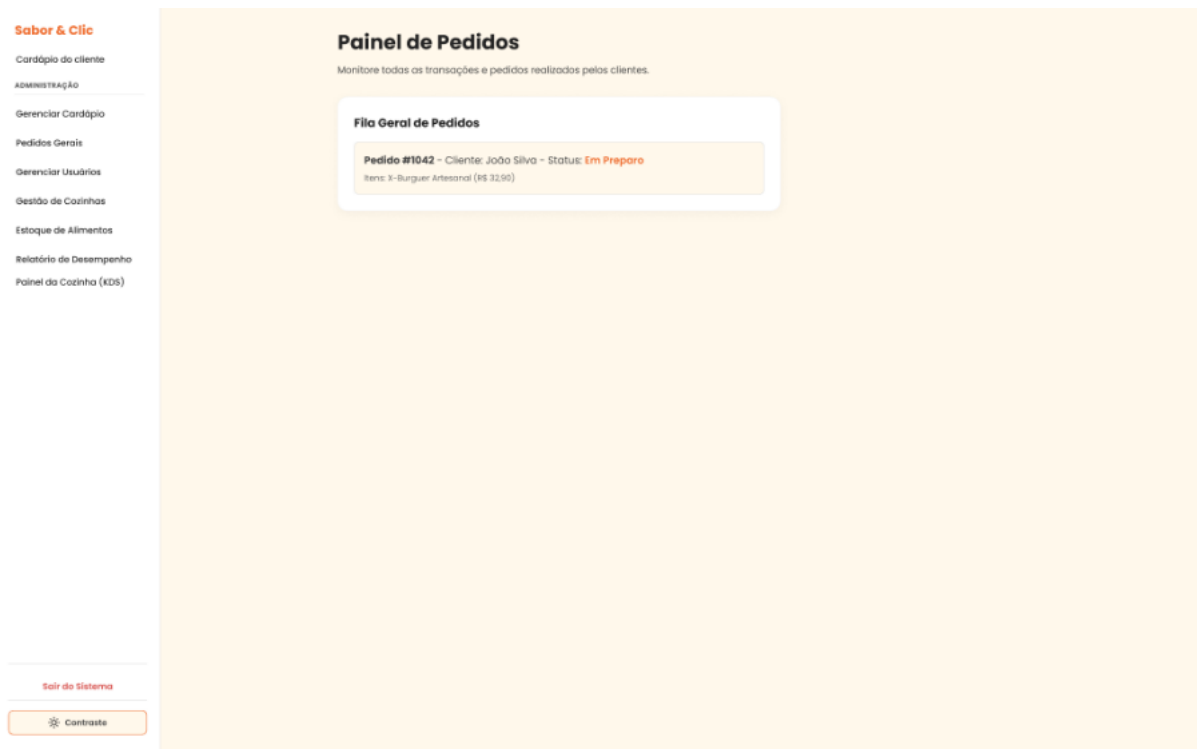


Figura: Página do painel de pedidos

Fonte: Elaborado pela Equipe, 2026

11.3.5 Relatório de desempenho

Página responsável pela emissão e visualização de métricas e indicadores globais, permitindo acompanhar o desempenho geral das operações e auxiliar na análise dos resultados do negócio.

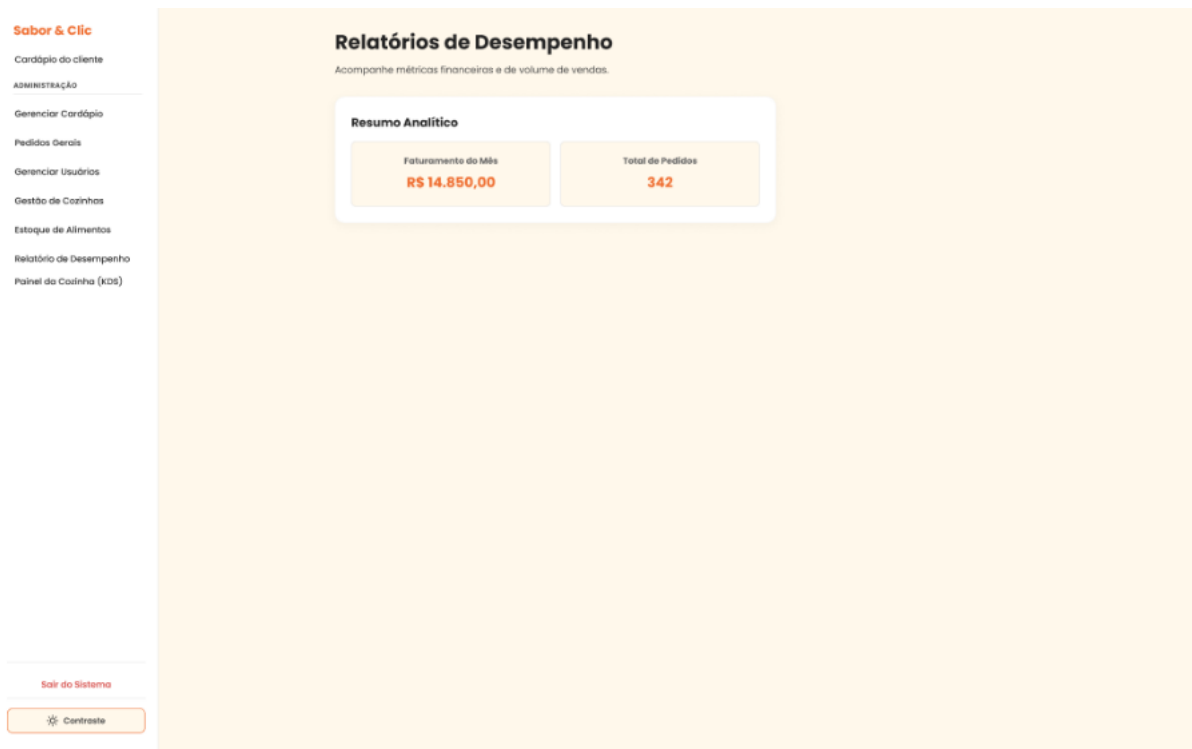


Figura: Página do relatório de desempenho

Fonte: Elaborado pela Equipe, 2026

11.3.6 Gestão de usuários

Página responsável pela administração de permissões e perfis de acesso, permitindo definir quais funcionalidades e informações cada tipo de usuário pode acessar dentro do sistema.

Sabor & Clic

Cardápio do cliente

ADMINISTRAÇÃO

Gerenciar Cardápio

Pedidos Gerais

Gerenciar Usuários

Gestão de Cozinhas

Estoque de Alimentos

Relatório de Desempenho

Panel da Cozinha (KDS)

Sair do Sistema

Contraste

Gerenciamento de Usuários

Cadastre novos administradores, cozinheiros ou clientes.

Adicionar Novo Usuário

Nome Completo

E-mail

Cargo / Função

Administrador

Cadastrar Usuário

Usuários do Sistema

Carlos Admin	Administrador	admin@saborclic.com	Remover
Ana Cozinha	Cozinheira	cozinha@saborclic.com	Remover
Pedro Cliente	Cliente	cliente@saborclic.com	Remover

Figura: Página do gerenciamento de usuários

Fonte: Elaborado pela Equipe, 2026

11.3.7 Cardápio do administrador

O administrador tem acesso a uma página de cardápio própria do seu módulo, permitindo consultar os itens disponíveis e suas respectivas informações, porém sem a possibilidade de realizar compras ou efetuar pedidos pelo sistema.

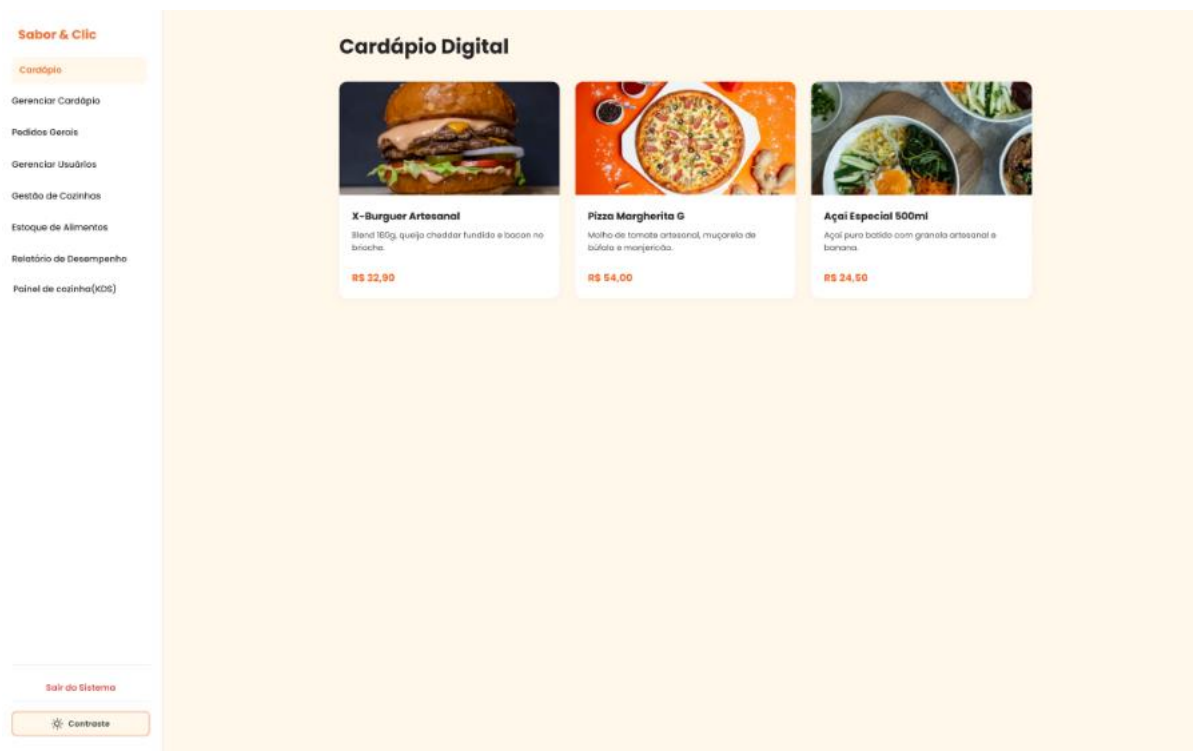


Figura: Página do cardápio do administrador

Fonte: Elaborado pela Equipe, 2026

11.3.8 Exemplo de uma página em Alto Contraste

A versão em alto contraste foi desenvolvida para melhorar a acessibilidade e facilitar a visualização dos elementos do site. Ela utiliza maior diferença entre as cores de fundo, textos e botões, tornando as informações mais claras e fáceis de identificar durante a navegação.

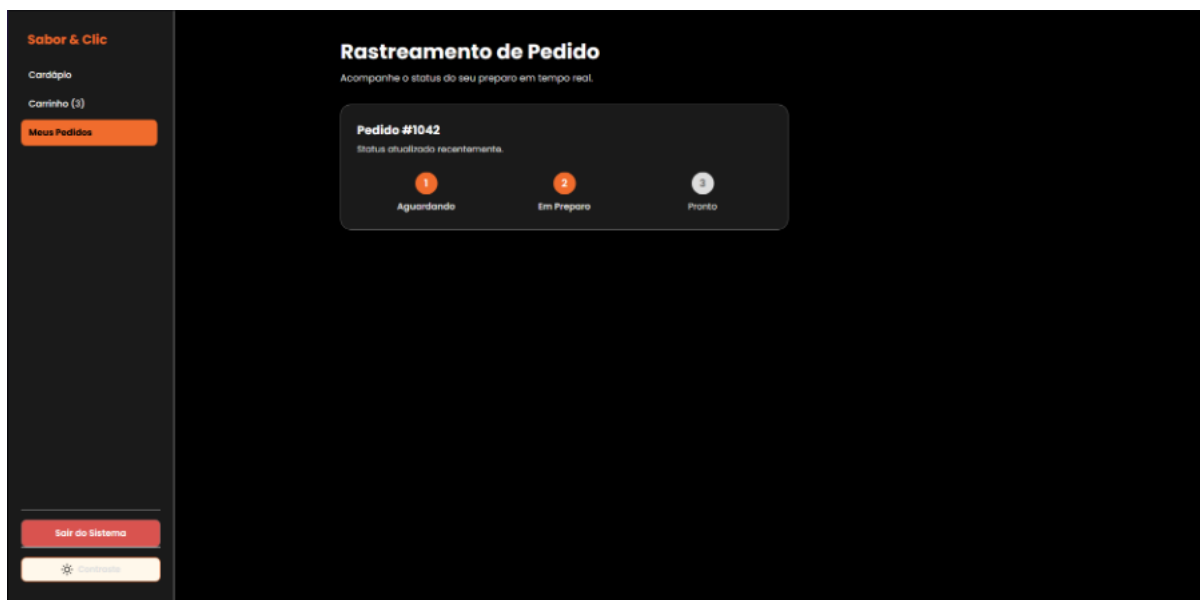


Figura: página de rastreamento do pedido

Fonte: Elaborado pela Equipe, 2026

12 Arquitetura Flask

Nesta etapa, é apresentada a estrutura da aplicação desenvolvida com Flask, responsável por organizar e disponibilizar as funcionalidades do sistema por meio de suas rotas. O arquivo `app.py` atua como ponto central da aplicação, realizando a configuração do Flask e o registro dos Blueprints responsáveis por diferentes áreas, como usuários, pratos, pedidos e cozinhas. A seguir, são apresentados trechos do código que demonstram essa organização e o funcionamento inicial da aplicação.

Trecho 1: `app.py`

```
from flask import Flask
from routes.usuarios import usuarios_bp
from routes.pratos import pratos_bp
from routes.pedidos import pedidos_bp
from routes.cozinhas import cozinhas_bp

app = Flask(__name__)

app.register_blueprint(usuarios_bp,
url_prefix="/api/usuarios")
app.register_blueprint(pratos_bp, url_prefix="/api/pratos")
app.register_blueprint(pedidos_bp, url_prefix="/api/pedidos")
```

```

app.register_blueprint(cozinhas_bp,
url_prefix="/api/cozinhas")

@app.route("/")
def inicio():
    return {
        "sistema": "Sabor & Clic",
        "status": "online"
    }

@app.route("/api")
def api():
    return {
        "mensagem": "API Sabor & Clic",
        "rotas": {
            "usuarios": "/api/usuarios",
            "pratos": "/api/pratos",
            "pedidos": "/api/pedidos",
            "cozinhas": "/api/cozinhas"
        }
    }

if __name__ == "__main__":
    app.run(debug=True)

```

O código a seguir apresenta uma das estruturas de rotas Flask já desenvolvidas como base para o projeto, neste caso, relacionada ao setor de cozinhas. Para organizar essas funcionalidades, foi utilizado o Blueprint `cozinhas_bp`, responsável por agrupar as rotas referentes às cozinhas e facilitar a organização da aplicação.

Entre as operações implementadas estão a listagem das cozinhas, o cadastro de uma nova cozinha e a alteração de sua disponibilidade, utilizando diferentes métodos HTTP conforme a finalidade de cada operação. Neste momento, os dados são mantidos em uma estrutura em memória, servindo como uma base inicial para o desenvolvimento. Essa implementação ainda será adaptada e aprimorada durante a

Fase 2, quando as funcionalidades serão integradas de forma mais completa ao restante do sistema.

Trecho 2: Cozinha Rotas Flask

```
from flask import Blueprint, request, jsonify

cozinhas_bp = Blueprint("cozinhas", __name__)

cozinhas = []

@cozinhas_bp.route("/", methods=["GET"])
def listar_cozinhas():
    return jsonify(cozinhas)

@cozinhas_bp.route("/", methods=["POST"])
def cadastrar_cozinha():

    dados = request.get_json()

    nome = dados.get("nome")
    endereco = dados.get("endereco")

    if not nome or not endereco:
        return jsonify({
            "erro": "Nome e endereço são obrigatórios"
        }), 400

    cozinha = {
        "id": len(cozinhas) + 1,
        "nome": nome,
        "endereco": endereco,
        "disponivel": True
    }

    cozinhas.append(cozinha)
```

```

        return jsonify(cozinha), 201

@cozinhas_bp.route("/<int:id>/disponibilidade",
methods=["PUT"])
def alterar_disponibilidade(id):

    cozinha = next(
        (c for c in cozinhas if c["id"] == id),
        None
    )

    if cozinha is None:
        return jsonify({
            "erro": "Cozinha não encontrada"
        }), 404

    cozinha["disponivel"] = not cozinha["disponivel"]

    return jsonify(cozinha)

```

13 Arquitetura Javascript Inicial

Para estabelecer a base da lógica de interação da aplicação, foram desenvolvidas algumas estruturas iniciais em JavaScript, simulando funcionalidades que farão parte do sistema. A proposta é utilizar a linguagem não apenas para modificar elementos da interface, mas também para estabelecer regras de funcionamento e realizar validações antes que os dados sejam enviados para as demais partes da aplicação.

Como forma de demonstrar essa lógica, foi realizada uma simulação do processo de cadastro de um usuário. Para isso, foi considerado um formulário HTML contendo campos como nome, e-mail e senha. Esse formulário ainda não pertence à interface definitiva do projeto, tendo sido criado apenas como suporte para que fosse possível testar o comportamento do JavaScript e suas validações.

O código apresentado a seguir utiliza a manipulação do DOM para acessar os

campos do formulário e capturar os dados preenchidos. Também foram estabelecidos critérios para verificar se os campos foram preenchidos corretamente, se o nome possui um tamanho mínimo, se o e-mail apresenta uma estrutura básica válida e se a senha atende aos requisitos definidos. Caso sejam encontradas inconsistências, são apresentadas mensagens indicando os problemas; quando os dados estão corretos, o cadastro é simulado com uma mensagem de confirmação e a limpeza do formulário.

Trecho 3:

```
const formulario = document.getElementById("formulario");

function checarRequisitosSenha(senha) {
    const erros = [];

    if (!/[A-Z]/.test(senha)) erros.push("Falta uma letra maiúscula");
    if (!/[0-9]/.test(senha)) erros.push("Falta um número");
    if (!/^[^a-zA-Z0-9]/.test(senha)) erros.push("Falta um caractere especial");

    return {erros: erros};
}

formulario.addEventListener("submit", function(event) {
    event.preventDefault();

    const nome = document.getElementById("nome").value.trim();
    const email = document.getElementById("email").value.trim();
    const senha = document.getElementById("senha").value.trim();

    let erros = [];

    if (nome === "" || email === "" || senha === "") {
        erros.push("Preencha todos os campos!");
    }
});
```

```

    } else {
        if (nome.length < 4) {
            erros.push("Nome muito curto!");
        }
        if (!email.includes("@")) {
            erros.push("Digite um e-mail válido!");
        }
        if (senha.length < 8) {
            erros.push("Senha muito pequena (mínimo 8
caracteres)!");
        }

        const requisitosSenha = checarRequisitosSenha(senha);
        erros = erros.concat(requisitosSenha.erros);
    }

    if (erros.length > 0) {
        alert(erros.join("\n"));
        return;
    } else{
        alert("Formulário enviado com sucesso!");
        formulario.reset();
    }

});

```

Essa implementação representa uma primeira aproximação da lógica que será utilizada na aplicação, permitindo testar as regras de interação mesmo antes de a interface definitiva estar concluída. Posteriormente, essas estruturas poderão ser integradas às páginas reais do sistema e às demais tecnologias utilizadas no projeto, ampliando suas funcionalidades durante a Fase 2.

14 Visualização sobre construção do Sabor&Clic na API

A construção do Sabor & Clic na API envolve a integração entre JavaScript e

Python com Flask, permitindo que a interface do sistema se comunique com o servidor e que as funcionalidades sejam processadas de maneira organizada. Nesta parte do projeto, são apresentados recursos que poderão ser utilizados na implementação das funcionalidades previstas, considerando tanto a manipulação da interface quanto o processamento e armazenamento dos dados.

14.1 Recursos do JavaScript

No JavaScript, serão utilizados recursos voltados principalmente à manipulação do DOM, controle de eventos, validação de dados e comunicação com a API. Esses recursos permitem que a interface responda às ações do usuário de forma dinâmica, evitando que cada alteração ou operação realizada exija uma nova página inteira.

Um dos principais recursos previstos é o `fetch()`, utilizado para realizar requisições HTTP de forma assíncrona. Por meio dele, o JavaScript poderá enviar ou solicitar informações à API sem a necessidade de recarregar toda a página. Isso será importante, por exemplo, para cadastrar informações, consultar dados ou atualizar determinada parte da interface, tornando a utilização do sistema mais dinâmica e fluida.

Também poderão ser utilizadas diferentes estruturas de repetição, como `for`, `for...of` e `forEach()`, para percorrer listas de dados recebidas da API e trabalhar com diferentes registros. Esses recursos poderão ser aplicados, por exemplo, na exibição de pratos, ingredientes, pedidos ou outros elementos que precisem ser apresentados de forma dinâmica.

Outro recurso importante será o `event.preventDefault()`, utilizado no controle de eventos de formulários. Ele permite impedir o comportamento padrão de um formulário, como o recarregamento da página após seu envio, possibilitando que o JavaScript faça a validação dos dados e, posteriormente, realize a comunicação com a API de maneira assíncrona.

A manipulação do DOM também terá papel fundamental. Recursos como `document.getElementById()`, `querySelector()` e `createElement()` poderão ser utilizados para localizar elementos existentes, criar novos elementos e atualizar o conteúdo da página conforme as informações recebidas. Dessa maneira, partes da interface poderão ser construídas ou modificadas diretamente pelo JavaScript, permitindo que os dados do sistema sejam refletidos na tela sem a necessidade de reconstruir toda a

página.

Além disso, estruturas condicionais, funções, arrays, objetos e métodos de manipulação de dados poderão ser combinados para controlar o estado da interface e representar informações temporárias durante a utilização do sistema. Esses recursos formam a base para que, posteriormente, o JavaScript possa trabalhar de maneira integrada às rotas e aos dados disponibilizados pela API.

14.2 Recursos do Python e Flask

No lado do servidor, o Python, juntamente com o Flask, será responsável por estruturar a API, receber requisições, processar os dados e retornar respostas para o JavaScript. Como referência para essa construção, podem ser observadas estruturas presentes na aplicação Supermercado Online, que apresenta uma organização baseada em rotas, controllers, models e uma camada responsável pelo banco de dados. Essa estrutura serve como uma referência para a forma como determinados recursos poderão ser empregados no Sabor & Clic, ainda que a aplicação atual possua funcionalidades e necessidades diferentes.

O Flask poderá ser utilizado para criação e organização das rotas da API, recebendo diferentes métodos HTTP de acordo com a operação realizada. Recursos como request permitirão acessar dados enviados pelo cliente, enquanto jsonify poderá ser utilizado para retornar informações em formato JSON. A utilização de Blueprints também permitirá separar as rotas de acordo com as funcionalidades do sistema, mantendo uma estrutura mais organizada à medida que a aplicação crescer.

Funções do Flask como `render_template`, `redirect`, `url_for` e `session` também poderão fazer parte da construção das funcionalidades relacionadas ao servidor. O `render_template` permite apresentar páginas a partir de arquivos HTML, enquanto `redirect` e `url_for` podem auxiliar no direcionamento entre diferentes rotas. Já `session` poderá ser utilizada para manter determinadas informações relacionadas ao estado do usuário durante sua navegação pela aplicação.

O Flask permitirá a criação e organização das rotas da API, utilizando diferentes métodos HTTP de acordo com a operação realizada. Entre eles, destaca-se o GET, que poderá ser utilizado para solicitar e consultar informações do sistema, como a listagem de pratos, pedidos ou cozinhas. Já o POST será utilizado quando for necessário enviar novos dados ao servidor, como no cadastro de um usuário, prato ou outro registro.

Para trabalhar com essas requisições, o Flask disponibiliza o recurso `request`, que permite acessar os dados enviados pelo cliente. Em uma requisição POST, por exemplo, o `request` poderá receber os dados encaminhados pelo JavaScript, permitindo que o servidor os leia, valide e posteriormente realize o processamento necessário. Assim, a comunicação poderá ocorrer de maneira organizada entre a interface e a API, com cada tipo de requisição sendo utilizado de acordo com a finalidade da operação.

Para o armazenamento dos dados, o `sqlite3`, disponível no próprio Python, poderá ser utilizado para estabelecer a comunicação com um banco de dados SQLite. A aplicação de referência demonstra uma estrutura na qual a conexão com o banco é centralizada e utilizada pelos diferentes componentes responsáveis pelos dados. No Sabor & Clic, uma organização semelhante poderá ser utilizada para armazenar informações como usuários, pratos, ingredientes, pedidos e demais dados necessários ao funcionamento do sistema. Embora poderemos ainda utilizar de outro recurso que não seja nativo do próprio python para a criação do banco de dados nesse projeto, o que dependerá do que a aplicação necessitará para sua melhor funcionalidade.

A biblioteca `openpyxl` também poderá ser empregada para a geração e manipulação de arquivos no formato Excel. Esse recurso pode ser útil em funcionalidades que envolvam relatórios ou exportação de informações, permitindo transformar dados registrados pelo sistema em planilhas para consulta e análise. Outro recurso que poderá ser utilizado é o `Path`, da biblioteca `pathlib`, para trabalhar com caminhos de arquivos e diretórios de maneira mais organizada. Ele pode auxiliar, por exemplo, na definição de locais para arquivos utilizados pela aplicação e na criação ou localização de diretórios necessários ao funcionamento do sistema.

Dessa forma, Python e Flask constituirão a base responsável pelo processamento da aplicação, enquanto o JavaScript ficará encarregado de proporcionar uma interação dinâmica com o usuário e de realizar a comunicação com a API. A combinação desses recursos permitirá que o Sabor & Clic evolua de suas estruturas iniciais para uma aplicação integrada, na qual as ações realizadas na interface possam ser processadas pelo servidor e refletidas nos dados do sistema.

15 Definição dos Níveis de Acesso e Regras de Validação de Dados

Como parte do planejamento da segurança do sistema, será adotado o modelo

RBAC (Role-Based Access Control), que permitirá definir os níveis de acesso de acordo com a função desempenhada por cada usuário. Serão considerados três níveis principais: Administrador, Cliente e Cozinheiro, cada um com diferentes permissões e restrições dentro da aplicação. O Administrador terá acesso às funções de gerenciamento do cardápio e demais atividades administrativas; o Cliente poderá visualizar os pratos e realizar pedidos; enquanto o Cozinheiro terá acesso às funcionalidades relacionadas ao acompanhamento e à atualização dos pedidos na cozinha. Essa definição inicial permitirá estabelecer quais ações estarão disponíveis ou bloqueadas para cada perfil, evitando que usuários tenham acesso a funcionalidades que não sejam necessárias à sua função.

Também serão estabelecidas regras de validação para os dados inseridos no sistema, definindo previamente quais características as informações deverão atender antes de serem processadas ou armazenadas. Entre essas regras, poderão ser considerados aspectos como o formato de um e-mail, o tamanho e preenchimento dos campos e o tipo de dado esperado para determinadas informações, como valores de pratos. Essas definições servirão como base para a implementação das validações nas etapas posteriores do projeto, contribuindo para evitar o recebimento de informações incorretas ou malformadas e para aumentar a segurança no tratamento dos dados.

16 Conclusões e Continuidade do Projeto

Concluimos, nesta primeira fase, a estruturação e o planejamento do Sabor & Clic, contemplando a identidade visual, o protótipo navegável, as telas destinadas aos três perfis de usuários, a organização do cardápio, o carrinho, a visualização de pedidos e a estrutura visual do KDS. Também foram desenvolvidos o DER, o MER e o dicionário de dados, além das estruturas iniciais em JavaScript e da arquitetura da API com Python e Flask, incluindo a organização das rotas por meio de Blueprints e o uso de dados simulados para representar algumas funcionalidades. A partir dessas definições, foram estabelecidas as bases para a integração entre interface, JavaScript, API e banco de dados.

Para a próxima fase, pretende-se avançar da estrutura conceitual e das simulações para a implementação integrada do sistema, conectando as páginas desenvolvidas à API e substituindo as estruturas utilizadas apenas para demonstração por funcionalidades efetivamente integradas. Também serão desenvolvidas as

operações necessárias para o gerenciamento dos dados, o armazenamento no banco de dados, a comunicação assíncrona entre JavaScript e API e os mecanismos de autenticação e controle de acesso. Dessa forma, a próxima etapa dará continuidade ao que foi planejado nesta fase, transformando as estruturas e definições já estabelecidas em uma aplicação funcional e integrada.

17 Referências

“BRModelo – Free Database ER Diagram Modeling Tool.” *BR Modelo*, 9 Jun. 2026, <https://brmodelo.org/>. Acesso em: 9 Sept. 2026.

Figma. “Figma: The Collaborative Interface Design Tool..” *Figma*, 2025, <https://www.figma.com>. Acesso em: 11 Set. 2026.

ALVES, Claudete De Oliveira. SPOWEB1 - Desenvolvimento Web 1. 2026. Ambiente Virtual de Aprendizagem (Moodle Câmpus). Coordenadoria de EaD, Instituto Federal de São Paulo (IFSP), Câmpus São Paulo. Disponível em: <<https://eadcampus.spo.ifsp.edu.br/course/view.php?id=9526>>. Acesso em: 13 set. 2026.

Sabor&Clic. Repositorio PROJETO-INTEGRADOR-SABOR-CLIC. GitHub, 2026. Disponível em: <https://github.com/NicolasGallina01/PROJETO-INTEGRADOR-SABOR-CLIC>